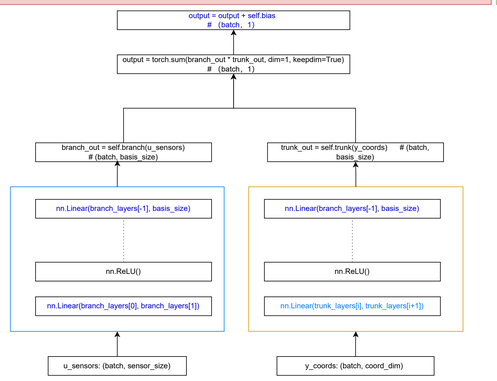
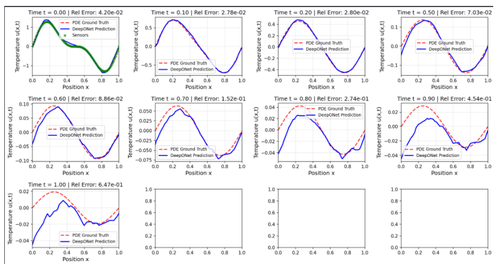
Neural_Operators神经算子

DeepOnet

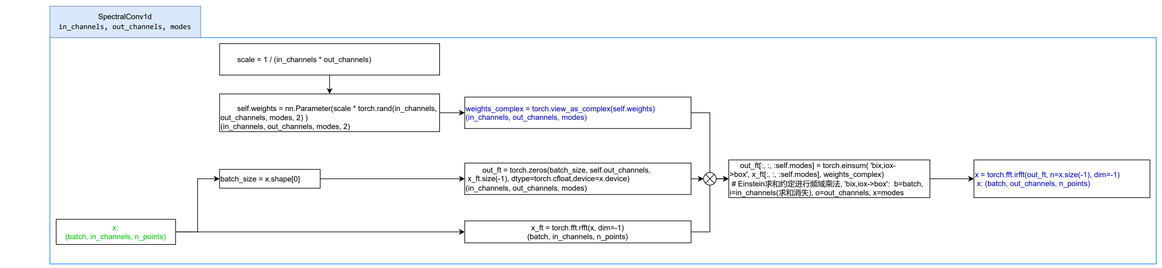


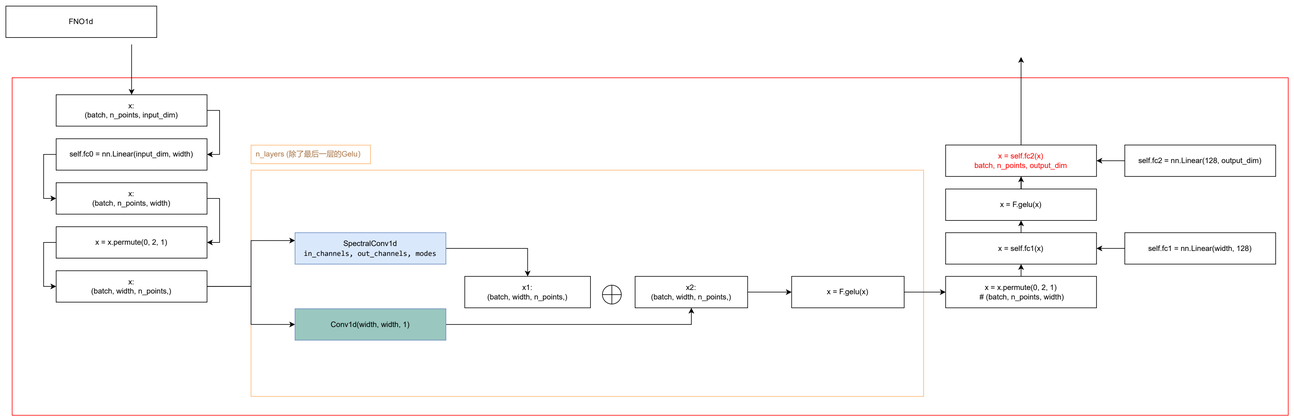
https://github.com/lululxvi/deepxde

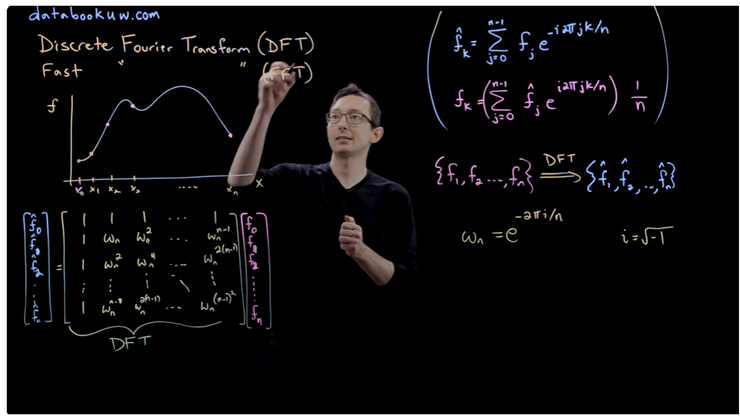
```
"""
完整的DeepONet实现：1D热传导方程算子学习
问题：学习从初始温度分布 u(x,0) 到任意时刻 t 的温度分布 u(x,t) 的映射
"""
```



FNO








```
FNO的本质：

y = F⁻¹[Truncate(F[x]) ⊙ W_learnable]
        ↑                ↑
    只保留低频        可学习变换

其中：
- F: 傅里叶变换（FFT）
- F⁻¹: 逆傅里叶变换（IFFT）
- Truncate: 只保留前modes个频率
- ⊙: 频域中的线性变换（einsum）
- W: 可学习的权重矩阵
```

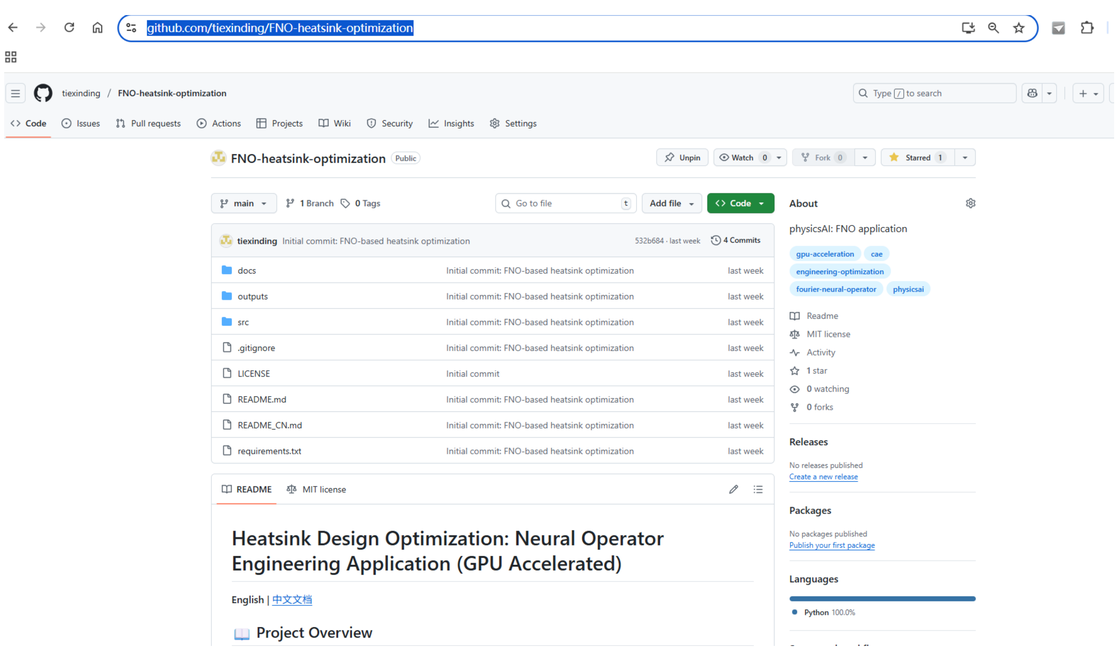
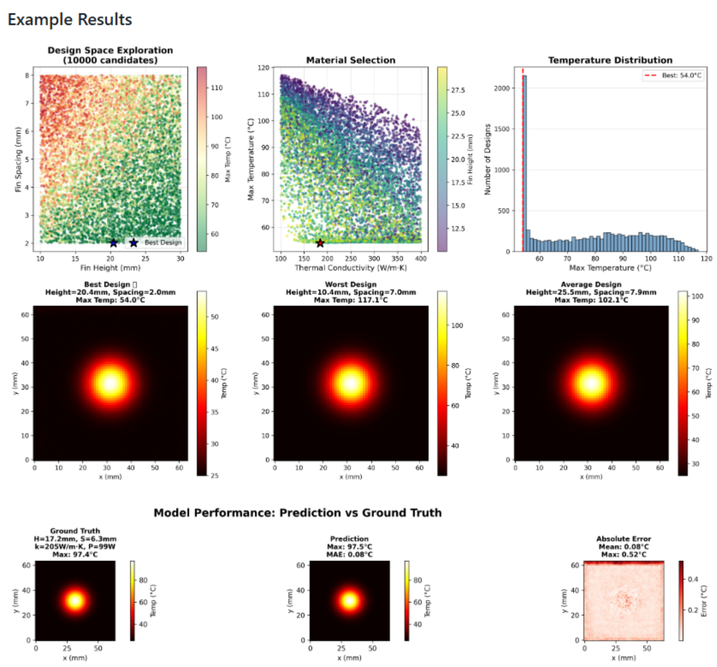
```
FFT基础操作:
n = 128
t = np.linspace(0, 1, n, endpoint=False)
freq = 5
x = np.sin(2 * np.pi * freq * t + np.pi/4) # 添加初相位
x_fft = np.fft.rfft(x)
freqs = np.fft.rfftfreq(n, d=1/n)
#d is the sampling interval (inverse of the sampling rate)
#f = [0, 1, ..., n/2-1, n/2] / (d*n)
amplitude = np.abs(x_fft)
phase = np.angle(x_fft)
phase_deg = np.angle(x_fft, deg=True)
real_part = np.real(x_fft)
imag_part = np.imag(x_fft)
x_recovered = np.fft.irfft(x_fft, n=n)
```

https://github.com/neuraloperator/neuraloperator

SpectralConv2d
in_channels, out_channels, modes

```
class SpectralConv2d(nn.Module):
    """
    真正的傅里叶层（频域卷积）
    GPU优化：FFT操作在GPU上非常快
    """

    def __init__(self, in_channels, out_channels, modes):
        super().__init__()
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.modes = modes

        # 频域权重（复数）
        scale = 1 / (in_channels * out_channels)
        self.weights = nn.Parameter(
            scale * torch.rand(in_channels, out_channels, modes, modes, 2)
        )

    def forward(self, x):
        """
        GPU优化：FFT在GPU上比CPU快10-100倍

        修复：FFT操作必须用FP32，不能用FP16（混合精度）
        """
        batch, channels, H, W= x.shape

        # 保存原始dtype
        dtype = x.dtype

        # ===== 关键修复：FFT强制使用FP32 =====
        if x.dtype == torch.float16:
            x = x.float()  # FP16 → FP32

        # 1. FFT到频域（GPU加速）
        x_ft = torch.fft.rfft2(x, norm='ortho')

        # 2. 频域卷积（保留低频modes）第二维度具备对称性
        out_ft = torch.zeros(batch, self.out_channels, H, W//2+1,
                             dtype=torch.cfloat, device=x.device)

        # 只在低频进行卷积
        out_ft[:, :, :self.modes, :self.modes] = torch.einsum(
            "bixy,ioxy->boxy",
            x_ft[:, :, :self.modes, :self.modes],
            torch.view_as_complex(self.weights)
        )

        # 3. IFFT回空间域（GPU加速）
        x = torch.fft.irfft2(out_ft, s=(H, W), norm='ortho')

        # 恢复原始dtype（如果需要）
        if dtype == torch.float16:
            x = x.half()  # FP32 → FP16

        return x
```

FNO2d

```
class TrueFNO(nn.Module):
    """
    真正的FNO实现（GPU优化版）

    GPU优化要点：
    1. 所有操作在GPU上
    2. BatchNorm利用GPU并行
    3. 避免CPU-GPU数据传输
    """

    def __init__(self, param_dim=4, grid_size=64, width=32, modes=12):
        super().__init__()
        self.grid_size = grid_size
        self.width = width
        self.modes = modes

        # 1. 参数编码器（GPU加速矩阵乘法）
        self.param_encoder = nn.Sequential(
            nn.Linear(param_dim, 64),
            nn.GELU(),
            nn.LayerNorm(64),
            nn.Linear(64, 128),
            nn.GELU(),
            nn.LayerNorm(128),
            nn.Linear(128, width * grid_size * grid_size)
        )

        # 2. FNO层（频域 + 空间域）
        self.spectral_conv1 = SpectralConv2d(width, width, modes)
        self.spectral_conv2 = SpectralConv2d(width, width, modes)
        self.spectral_conv3 = SpectralConv2d(width, width, modes)
        self.spectral_conv4 = SpectralConv2d(width, width, modes)

        # 局部卷积（GPU上卷积很快）
        self.local_conv1 = nn.Conv2d(width, width, 1)
        self.local_conv2 = nn.Conv2d(width, width, 1)
        self.local_conv3 = nn.Conv2d(width, width, 1)
        self.local_conv4 = nn.Conv2d(width, width, 1)

        # BatchNorm（GPU并行优化）
        self.bn1 = nn.BatchNorm2d(width)
        self.bn2 = nn.BatchNorm2d(width)
        self.bn3 = nn.BatchNorm2d(width)
        self.bn4 = nn.BatchNorm2d(width)

        # 3. 输出解码器
        self.decoder = nn.Sequential(
            nn.Conv2d(width, 64, 3, padding=1),
            nn.GELU(),
            nn.BatchNorm2d(64),
            nn.Conv2d(64, 32, 3, padding=1),
            nn.GELU(),
            nn.BatchNorm2d(32),
            nn.Conv2d(32, 1, 1)
        )

    def forward(self, params):
        """
        前向传播（全程GPU）
        """
        batch = params.shape[0]

        # 参数 → 场
        x = self.param_encoder(params)
        x = x.view(batch, self.width, self.grid_size, self.grid_size)

        # FNO层1，频域分支，空间分支，残差连接
        x1 = self.spectral_conv1(x)
        x2 = self.local_conv1(x)
        x = self.bn1(x1 + x2 + x)
        x = F.gelu(x)

        # FNO层2
        x1 = self.spectral_conv2(x)
        x2 = self.local_conv2(x)
        x = self.bn2(x1 + x2 + x)
        x = F.gelu(x)

        # FNO层3
        x1 = self.spectral_conv3(x)
        x2 = self.local_conv3(x)
        x = self.bn3(x1 + x2 + x)
        x = F.gelu(x)

        # FNO层4
        x1 = self.spectral_conv4(x)
        x2 = self.local_conv4(x)
        x = self.bn4(x1 + x2 + x)
        x = F.gelu(x)

        # 解码
        temp = self.decoder(x).squeeze(1)

        return temp
```

https://github.com/tiexinding/FNO-heatsink-optimization